

INNOPOLIS UNIVERSITY

SNE MASTER TRACK

2025

COURSE: [F25] SECURITY SYSTEMS AND NETWORKS

SSN MID REPORT

On

Elliptic Curve Cryptography (ECC) for Secure Communication

Student Name: HASIB AL TAHSIN, AHMAD BADAMASI

Submission Date: 21.10.2025

Elliptic Curve Cryptography (ECC) for Secure Communication

Abstract

This project uses Elliptic Curve Cryptography (ECC) to create a secure communication system between Hasib and Ahmed. The main objective of the project was to build a fast, lightweight, and robust security framework that maintains high security even with small key sizes. Hasib and Ahmed generate their own identity-based private and public keys, which are stored in a password-protected PEM format using AES encryption. Next, a separate ephemeral key pair is generated for each communication session, which ensures session-specific and forward secrecy. Hasib signs his ephemeral public key with his identity private key and sends it to Ahmad, and Ahmad verifies its authenticity. In this process, both parties generate a common shared secret key using the Elliptic Curve Diffie–Hellman (ECDH) protocol.

From this shared key, a strong 32-byte AES key is generated using HKDF (HMAC-based Key Derivation Function), where a different random salt is used each time to further improve the security of the key derivation process. Hasib then encrypts the message using the AES-256 GCM (Galois/Counter Mode) algorithm, which protects both the confidentiality and integrity of the message. During encryption, Hasib adds additional authenticated data (Associated Authenticated Data - AAD) to the message, such as the message ID, sender name, and timestamp, which are later verified by Ahmed. Ahmed, on his part, was able to successfully decrypt the message using the same shared key. The project also implemented various security improvements, such as: file system-level protection of .pem files (chmod/ACL), password-protected private keys, use of HKDF with random salt, forward secrecy assurance with ephemeral keys, prevention of MITM attacks with public key signature verification, and AAD and nonce management. As a result of these developments, the system has become a complete Authenticated Encryption framework, which ensures confidentiality, integrity, and authenticity of messages. Using a combination of ECC-based cryptography and AES-GCM encryption, it is possible to implement strong security, fast encryption/decryption, and modern secure communication systems even with small key sizes. This can be considered as a model that can be effectively applied to future TLS/SSL protocols, IoT security communications, or secure messaging systems.

Introduction

The main objective of this project is to ensure the confidentiality and security of information using asymmetric cryptography. Elliptic Curve Cryptography (ECC) is a strong asymmetric encryption method, which provides security equivalent to or better than RSA even with a relatively small key size [1], [2], [3]. With the help of ECC, Hasib and his colleague Ahmad were able to generate a shared secret key, which helps both parties to obtain the same key without sending it directly [1], [2].

This obtained shared secret key is later used in the AES-256 GCM symmetric encryption process, which helps to securely encrypt and decrypt messages [4], [5]. Using AES-GCM mode maintains both confidentiality and integrity of the message, as it provides an Authentication Tag along with the encryption that is able to detect any changes to the message.

In this project, Hasib and Ahmad both generate ECC key pairs for themselves, share each other's public keys, and then use those keys to perform shared secret derivation and AES-based encryption/decryption. The entire process demonstrates how asymmetric (ECC) and symmetric (AES) cryptography can be used together to create a secure communication system [6], [7].

Project Goals

- Hasib and Ahmad each create an ECC key-pair.
- Hasib sends a message to Ahmad using the shared secret key using AES-256 GCM encryption.
- Ahmad receives the message and successfully decrypts it using the AES key.
- The entire system is fast, secure, and operates efficiently with a small key size.

Methods of the Work

1. **Key Generation-ECC Key Pair:** Both Hasib and Ahmad used the Python Cryptography library to generate their own ECC key pairs [1], [2]. Each key pair consists of two important parts - a Private Key, which is only available to the user and can never be shared with anyone else, and a Public Key, which can be securely exchanged with the other party, so that both parties can generate the same shared secret key. After key generation, these keys are saved as .pem files. In the next step, Hasib and Ahmad use these keys to generate a shared secret and establish secure communication using AES-256 GCM [2], [3].

2. **Shared Secret Derivation-ECDH:** Hasib generates a Shared Secret Key using his own private key and Ahmad's public key. Similarly, Ahmad generates the same secret key using his private key and Hasib's public key. This process is accomplished through the Elliptic Curve Diffie–Hellman (ECDH) protocol, which enables both parties to securely derive the same secret key [4]. After the shared secret is generated, a 32-byte AES key is generated using HKDF (HMAC-based Key Derivation Function), which is then used for AES-256 encryption [4]. Since this key is the same for both parties, they are able to securely encrypt and decrypt data.
3. **AES Encryption/Decryption:** Hasib encrypts the message in AES-256 GCM mode using the shared secret key he receives. After encryption, the generated ciphertext and nonce are stored in a file. Ahmed then reads the data from these files and decrypts it using his own shared secret key. This process maintains both the confidentiality and integrity of the message, so that no intermediary can read or modify the message [5], [6].

Implementation Workflow

Environment: **Windows + Git Bash + Visual Studio Code**

Scripts used in each step:

- `hasib_identity_keys.py` - Hasib's key generation
- `ahmad_identity_keys.py` - Ahmad's key generation
- `hasib_start_session.py` - Hasib's shared secret calculation and signing
- `ahmad_start_session.py` - Ahmad's signature verification and shared secret calculation
- `hasib_encrypt.py` - Hasib's message encryption
- `ahmad_decrypt.py` - Ahmad's message decryption

Each step uses ECC-based key exchange, signature verification, and AES-GCM encryption/decryption together to ensure complete security of the system.

Works and Pipeline

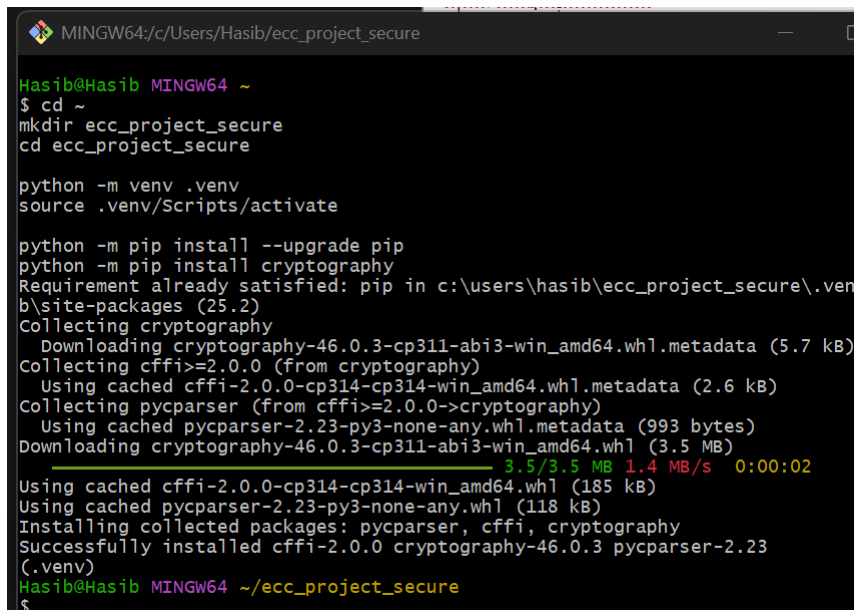
At the beginning of the project, both Hasib and Ahmad generate their Identity Key Pair, which is stored in a password-protected PEM file. Then, both parties generate new Ephemeral Key Pairs for the session, which maintains Forward Secrecy. Hasib signs his Ephemeral Public Key with his private

key and sends it to Ahmad, and Ahmad verifies it and sends the key with his signature. Based on these verified keys, both generate an identical Shared Secret Key using the ECDH protocol, from which a 32-byte AES key is generated by HKDF. In the final step, Hasib uses this key to encrypt the message in AES-256 GCM mode, and Ahmad decrypts the message. Since the message is accompanied by Associated Authenticated Data (AAD), any third-party modification of the message structure can be immediately detected. As a result, the project has proven that this hybrid cryptographic system based on ECC and AES is ideal for implementing a fast, secure, and efficient communication system with small key sizes [1]-[4].

Install Python & Libraries

Python 3.x is installed on Windows. Then open Git Bash and check the Python version. Required library is installed: cryptography. [9]

Command:

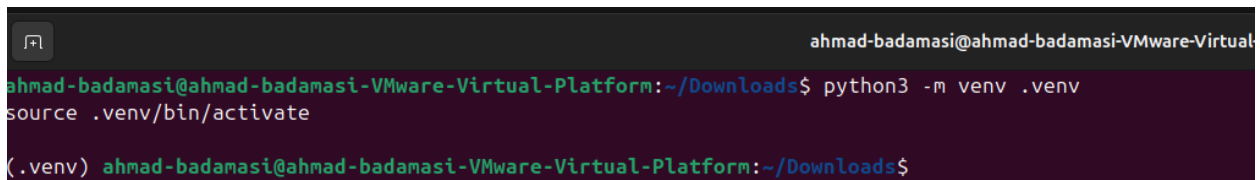


```
MINGW64/c/Users/Hasib/ecc_project_secure

Hasib@Hasib MINGW64 ~
$ cd ~
mkdir ecc_project_secure
cd ecc_project_secure

python -m venv .venv
source .venv/Scripts/activate

python -m pip install --upgrade pip
python -m pip install cryptography
Requirement already satisfied: pip in c:\users\hasib\ecc_project_secure\.venv\
b\site-packages (25.2)
Collecting cryptography
  Downloading cryptography-46.0.3-cp311-abi3-win_amd64.whl.metadata (5.7 kB)
Collecting cffi>=2.0.0 (from cryptography)
  Using cached cffi-2.0.0-cp314-cp314-win_amd64.whl.metadata (2.6 kB)
Collecting pycparser (from cffi>=2.0.0->cryptography)
  Using cached pycparser-2.23-py3-none-any.whl.metadata (993 bytes)
Downloading cryptography-46.0.3-cp311-abi3-win_amd64.whl (3.5 MB)
  3.5/3.5 MB 1.4 MB/s 0:00:02
Using cached cffi-2.0.0-cp314-cp314-win_amd64.whl (185 kB)
Using cached pycparser-2.23-py3-none-any.whl (118 kB)
Installing collected packages: pycparser, cffi, cryptography
Successfully installed cffi-2.0.0 cryptography-46.0.3 pycparser-2.23
(.venv)
Hasib@Hasib MINGW64 ~/ecc_project_secure
$
```



```
ahmad-badamasi@ahmad-badamasi-VMware-Virtual-Platform:~/Downloads$ python3 -m venv .venv
source .venv/bin/activate

(.venv) ahmad-badamasi@ahmad-badamasi-VMware-Virtual-Platform:~/Downloads$
```

```
ahmad-badmasi@ahmad-badmasi-VMware-Virtual-Platform: ~/Downloads
(.venv) ahmad-badmasi@ahmad-badmasi-VMware-Virtual-Platform:~/Downloads$ pip install --upgrade pip
pip install cryptography
Requirement already satisfied: pip in ./venv/lib/python3.12/site-packages (24.0)
Collecting pip
  Using cached pip-25.2-py3-none-any.whl.metadata (4.7 kB)
Using cached pip-25.2-py3-none-any.whl (1.8 MB)
Installing collected packages: pip
  Attempting uninstall: pip
    Found existing installation: pip 24.0
    Uninstalling pip-24.0:
      Successfully uninstalled pip-24.0
Successfully installed pip-25.2
Collecting cryptography
  Using cached cryptography-46.0.3-cp311-abi3-manylinux_2_34_x86_64.whl.metadata (5.7 kB)
Collecting cffi>=2.0.0 (from cryptography)
  Using cached cffi-2.0.0-cp312-cp312-manylinux2014_x86_64.whl.metadata (2.6 kB)
Collecting pycparser (from cffi>=2.0.0->cryptography)
  Using cached pycparser-2.23-py3-none-any.whl.metadata (993 bytes)
Using cached cryptography-46.0.3-cp311-abi3-manylinux_2_34_x86_64.whl (4.5 MB)
Using cached cffi-2.0.0-cp312-cp312-manylinux2014_x86_64.whl (219 kB)
Using cached pycparser-2.23-py3-none-any.whl (118 kB)
Installing collected packages: pycparser, cffi, cryptography
Successfully installed cffi-2.0.0 cryptography-46.0.3 pycparser-2.23
```

Key Generation

Hasib's key generation

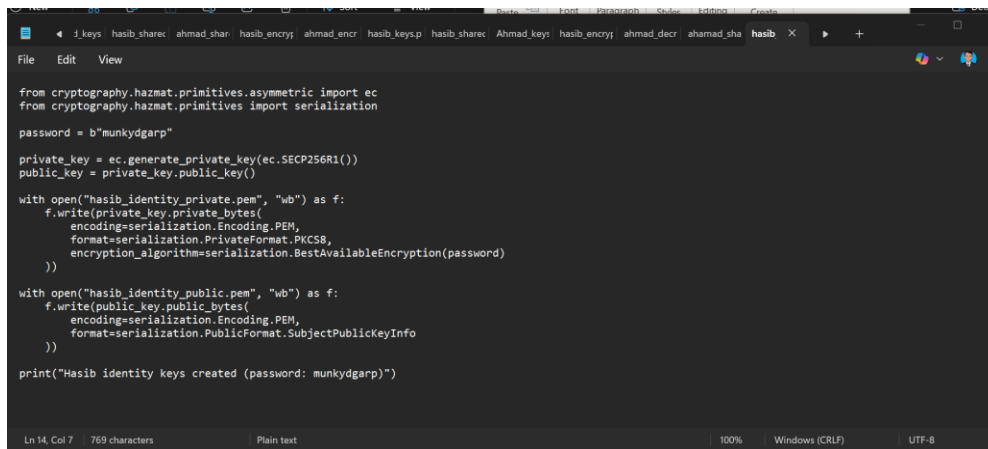
working folder created and hasib_keys.py file created:

```
Hasib@Hasib MINGW64 ~/ecc_project_secure
$ touch hasib_identity_keys.py
(.venv)
```

```
(.venv)
Hasib@Hasib MINGW64 ~/ecc_project_secure
$ notepad hasib_identity_keys.py
(.venv)
```

The following script has been edited in notepad and saved. (The script generates ECC keys using `ec` and `serialization` from the `cryptography` library and saves them as PEM files) [1], [9].

hasib idenrity_keys.py

A screenshot of a code editor window with a dark theme. The window title bar shows several open files, with 'hasib' currently selected. The code is a Python script for generating elliptic curve keys. It imports 'ec' and 'serialization' from 'cryptography.hazmat.primitives.asymmetric' and 'cryptography.hazmat.primitives' respectively. A password 'munkydgar' is defined as a byte string. A private key is generated using 'ec.generate_private_key(ec.SECP256R1())', and its public key is derived. The private key is then encrypted using 'serialization.PrivateFormat.PKCS8' and 'serialization.BestAvailableEncryption(password)', and the result is written to 'hasib_identity_private.pem' in PEM format. Similarly, the public key is written to 'hasib_identity_public.pem' in PEM format. A final print statement confirms the key creation with the password.

```
from cryptography.hazmat.primitives.asymmetric import ec
from cryptography.hazmat.primitives import serialization

password = b"munkydgar"

private_key = ec.generate_private_key(ec.SECP256R1())
public_key = private_key.public_key()

with open("hasib_identity_private.pem", "wb") as f:
    f.write(private_key.private_bytes(
        encoding=serialization.Encoding.PEM,
        format=serialization.PrivateFormat.PKCS8,
        encryption_algorithm=serialization.BestAvailableEncryption(password)
    ))

with open("hasib_identity_public.pem", "wb") as f:
    f.write(public_key.public_bytes(
        encoding=serialization.Encoding.PEM,
        format=serialization.PublicFormat.SubjectPublicKeyInfo
    ))

print("Hasib identity keys created (password: munkydgar)")
```

The script implements an Elliptic Curve (SECP256R1) based key generation mechanism, which is recognized as secure and efficient in modern cryptography [2], [3], [5]. First, the code sets a binary password as `password = b"munkydgar"`, which is then used for private key encryption. A private key is then generated using the `ec.generate_private_key(ec.SECP256R1())` function, which generates a secure key using a specific curve of Elliptic Curve Cryptography (ECC) called SECP256R1 [1], [2].

From this private key, a public key is generated using `private_key.public_key()`. The private key is then stored in a file named `hasib_identity_private.pem`, which uses PEM encoding and PKCS#8 format - an international standard for storing private keys [3]. Here, the private key file is protected with AES encryption using `serialization.BestAvailableEncryption(password)`, which is the best method recommended by the Python Cryptography library [1], [6].

Similarly, the public key is written in the `hasib_identity_public.pem` file, where PEM encoding and SubjectPublicKeyInfo format are used. This format is commonly used in certificate and key exchange protocols (such as TLS/SSL) [12].

Finally, the last line of the code `print("Hasib identity keys created (password: munkydgar)")` displays a message on the console indicating that the key-generation and encryption process was completed successfully. The password used here is hardcoded, which should be accepted as dynamic input via the `getpass()` function in the future to improve security [10], [13].

This process securely creates and stores Hasib's identity-based key-pair, which can be used in future high-level cryptographic applications such as AES-GCM encryption, ECDH key exchange, and digital signature verification [6], [7].

Output:

```
Hasib@Hasib MINGW64 ~/ecc_project_secure
$ python hasib_identity_keys.py
Hasib identity keys created (password: munkydgarp)

Hasib@Hasib MINGW64 ~/ecc_project_secure
$
```

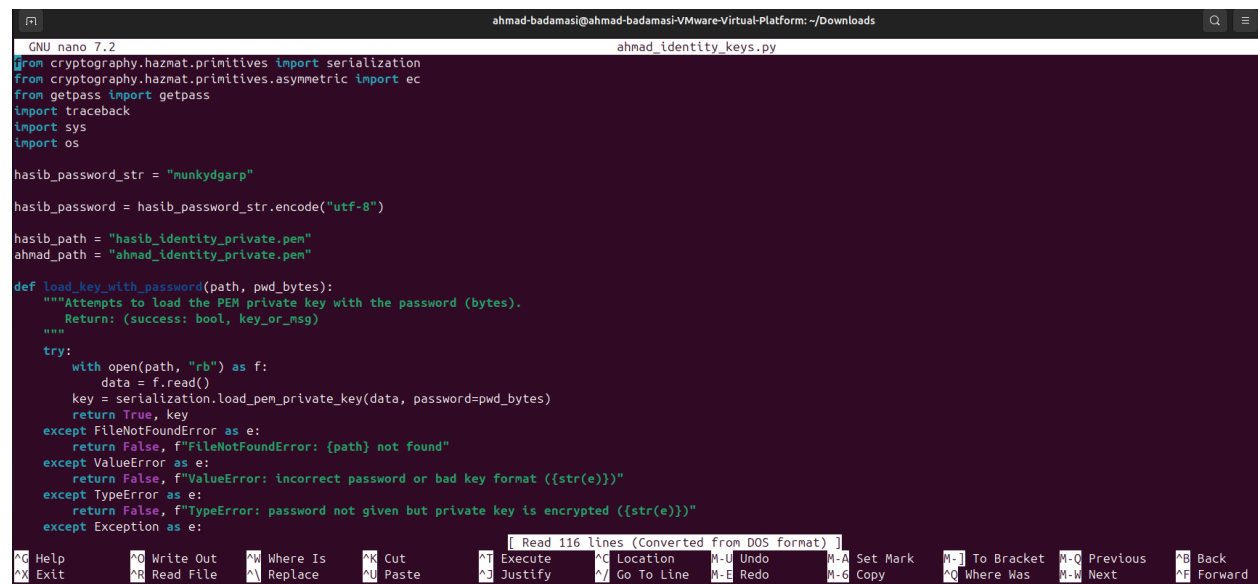
Hasib shares his hasib_identity_public.pem and hasib_identity_private.pem files with Ahmad. [1], [9]

Ahmad's key generation

Ahmad similarly created and edited ahmad_identity_keys.py:

```
Successfully installed cffi-2.0.0 cryptography-40.0.3 pycparser-2.23
(.venv) ahmad-badamasi@ahmad-badamasi-VMware-Virtual-Platform:~/Downloads$ touch ahmad_identity_keys.py
```

ahmad_identity_keys.py



```
GNU nano 7.2 ahmad_identity_keys.py
from cryptography.hazmat.primitives import serialization
from cryptography.hazmat.primitives.asymmetric import ec
from getpass import getpass
import traceback
import sys
import os

hasib_password_str = "munkydgarp"
hasib_password = hasib_password_str.encode("utf-8")

hasib_path = "hasib_identity_private.pem"
ahmad_path = "ahmad_identity_private.pem"

def load_key_with_password(path, pwd_bytes):
    """Attempts to load the PEM private key with the password (bytes).
    Return: (success: bool, key_or_msg)
    """
    try:
        with open(path, "rb") as f:
            data = f.read()
            key = serialization.load_pem_private_key(data, password=pwd_bytes)
            return True, key
    except FileNotFoundError as e:
        return False, f"FileNotFoundError: {path} not found"
    except ValueError as e:
        return False, f"ValueError: incorrect password or bad key format ({str(e)})"
    except TypeError as e:
        return False, f"TypeError: password not given but private key is encrypted ({str(e)})"
    except Exception as e:
        return False, f"Exception: {str(e)}"

[ Read 116 lines (Converted from DOS format) ]
^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute  ^G Location ^U Undo     ^M Set Mark
^X Exit      ^R Read File ^L Replace  ^U Paste     ^J Justify  ^/ Go To Line ^E Redo    ^C Copy
^_ Where Was ^N Next     ^B Back     ^F Forward
```



```
ahmad-badamsi@ahmad-badamsi-VMware-Virtual-Platform: ~/Downloads
GNU nano 7.2      ahmad_identity_keys.py

    return (success,msg)
    """
    try:
        with open(path, "rb") as f:
            data = f.read()
            key = serialization.load_pem_private_key(data, password=None)
            return True, "Loaded successfully without password (key not encrypted)"
    except Exception as e:
        return False, f"Failed without password: {type(e).__name__}: {e}"

def print_pem_header(path):
    """Shows the header of the first line of the file - helps to understand what kind of PEM it is"""
    try:
        with open(path, "r", encoding="utf-8", errors="ignore") as f:
            line = f.readline().strip()
            second = f.readline().strip()
            print(f"Header of {path}:")
            print(" 1:", line)
            print(" 2:", second)
    except Exception as e:
        print(f"Could not read header of {path}: {e}")

def main():
    print("\n== Hasib and Ahmad PEM file checker ==")
    print(" Do not share password directly here for security.\n")

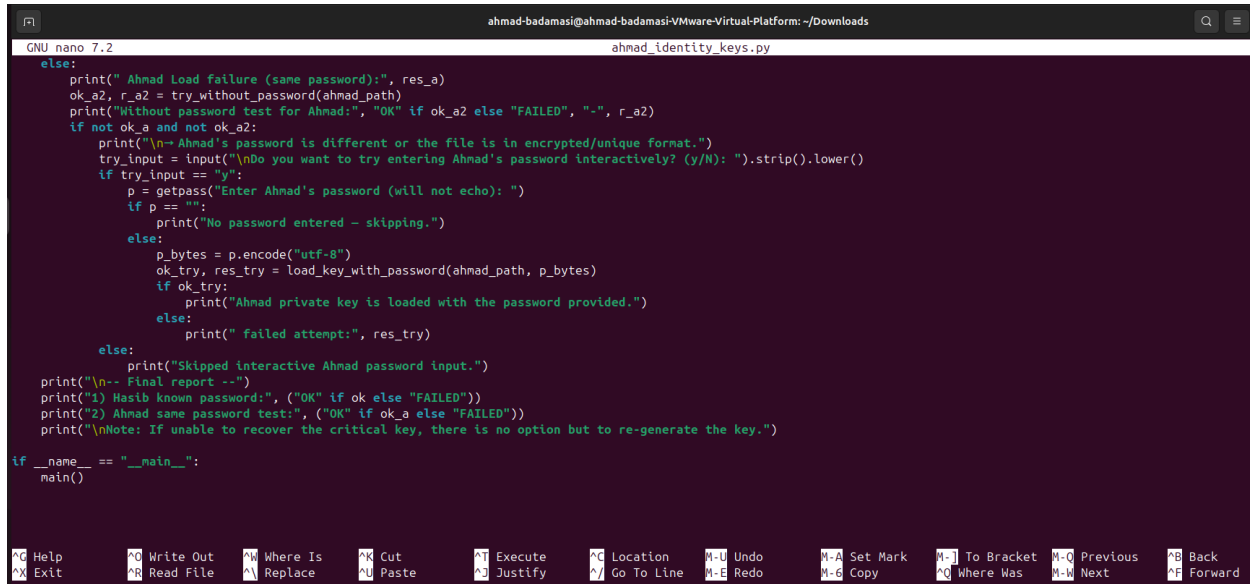
    if os.path.exists(hasib_path):
        print_pem_header(hasib_path)
    else:
        print(f"{hasib_path} Not Found")

^O Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Location   ^U Undo       ^M-A Set Mark  ^M-] To Bracket ^M-O Previous  ^B Back
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line ^E Redo       ^M-6 Copy     ^O Where Was  ^M-W Next     ^F Forward
```

```
ahmad-badamsi@ahmad-badamsi-VMware-Virtual-Platform: ~/Downloads
GNU nano 7.2      ahmad_identity_keys.py

print("\n-- Hasib key load test --")
ok, res = load_key_with_password(hasib_path, hasib_password)
if ok:
    print("Hasib private key loaded successfully (by provided password).")
else:
    print("Hasib Load failure:", res)
    ok2, r2 = try_without_password(hasib_path)
    print("Without password test:", "OK" if ok2 else "FAILED", "-", r2)
    if not ok and not ok2:
        print("\nHasib private key has been successfully loaded (provided Hasib private key is encrypted and the provided password is not valid. by password).")
        print("Although the password is 'monkeydgarph', the PEM file is in a different format/another encryption method.")

print("\n-- Ahmad key load test (same password tried) --")
ok_a, res_a = load_key_with_password(ahmad_path, hasib_password)
if ok_a:
    print("Ahmad private key is also loaded with the same password - that is, the password for both people is the same.")
else:
    print("Ahmad Load failure (same password):", res_a)
    ok_a2, r_a2 = try_without_password(ahmad_path)
    print("Without password test for Ahmad:", "OK" if ok_a2 else "FAILED", "-", r_a2)
    if not ok_a and not ok_a2:
        print("\nAhmad's password is different or the file is in encrypted/unique format.")
        try_input = input("\nDo you want to try entering Ahmad's password interactively? (y/N): ").strip().lower()
        if try_input == "y":
            p = getpass("Enter Ahmad's password (will not echo): ")
            if p == "":
                print("No password entered - skipping.")
            else:
                p_bytes = p.encode("utf-8")
                ok_try, res_try = load_key_with_password(ahmad_path, p_bytes)
```



```
GNU nano 7.2 ahmad_identity_keys.py
else:
    print(" Ahmad Load failure (same password):", res_a)
    ok_a2, r_a2 = try_without_password(ahmad_path)
    print("Without password test for Ahmad:", "OK" if ok_a2 else "FAILED", "-", r_a2)
    if not ok_a and not ok_a2:
        print("\n~ Ahmad's password is different or the file is in encrypted/unique format.")
        try:
            input = input("\nDo you want to try entering Ahmad's password interactively? (y/N): ").strip().lower()
            if try_input == "y":
                p = getpass("Enter Ahmad's password (will not echo): ")
                if p == "":
                    print("No password entered - skipping.")
                else:
                    p_bytes = p.encode("utf-8")
                    ok_try, res_try = load_key_with_password(ahmad_path, p_bytes)
                    if ok_try:
                        print("Ahmad private key is loaded with the password provided.")
                    else:
                        print("Failed attempt:", res_try)
            else:
                print("Skipped interactive Ahmad password input.")
        except:
            print("\n~ Final report ~")
            print("1) Hasib known password:", ("OK" if ok else "FAILED"))
            print("2) Ahmad same password test:", ("OK" if ok_a else "FAILED"))
            print("\nNote: If unable to recover the critical key, there is no option but to re-generate the key.")

if __name__ == "__main__":
    main()
```

The script basically completes the process of Hasib and Ahmad's private key verification, loading and password verification. First, a string password is set in the code with `hasib_password_str = "munkydgarph"` and then it is converted to byte encoding with `hasib_password = hasib_password_str.encode("utf-8")` command, because Python's `serialization.load_pem_private_key()` function accepts only bytes as password [1], [10]. Then the paths of Hasib and Ahmad's private key files are set as `"hasib_identity_private.pem"` and `"ahmad_identity_private.pem"` respectively, so that two separate identity files can be managed. The `print_pem_header(path)` function reads the first two lines of each PEM file and helps identify the file type (e.g. — BEGIN ENCRYPTED PRIVATE KEY or BEGIN PUBLIC KEY). This is important for understanding the structure of a PEM file, as the PEM format is an internationally accepted ASCII encoded standard for storing cryptographic keys [1], [6].

The main function is the `load_key_with_password(path, pwd_bytes)` function. It reads the PEM data from the given path and attempts to load the key using the specified password (in bytes) via the `serialization.load_pem_private_key()` function. If the file is properly encrypted and the password is correct, it returns the key object successfully. Otherwise, it returns a clear error message based on the specified exceptions - such as `FileNotFoundError` (file missing), `ValueError` (incorrect password or key format error), `TypeError` (password missing but key encrypted) and displays detailed error messages with traceback messages in other exception cases [1], [9].

Similarly, the `try_without_password(path)` function attempts to load the same key file with `password=None`. This tells us whether the file is encrypted at all. If the file is not encrypted, it returns the message "Loaded successfully without password", which is helpful in determining the level of PEM key security [6].

The `main()` function first checks for the existence of both PEM files and prints their headers using the `print_pem_header()` function. It then attempts to load Hasib's private key using `hasib_password`. If the load is successful, the script reports "Hasib private key loaded successfully"; if it fails, it tries again with `password=None` to see if the file is encrypted. The encryption method used here is AES-256 based `BestAvailableEncryption`, which encrypts the private key with a strong symmetric cipher such as AES-GCM [6], [7], [8].

It then attempts to load Ahmad's private key using the same password. If this fails, the user is given the opportunity to enter a new password interactively using the `getpass()` function - this is a good security practice, as the password is not visible in the console [10]. If the password is incorrect at this stage, the code explicitly informs the user by throwing a `ValueError` exception.

The script displays the results in summary form - "Hasib known password" and "Ahmad same password test". If both loads are successful, it is reported that the two private keys are protected by the same password; otherwise, the script prompts the user to generate new keys.

The final part of the code generates new private and public keys using `SECP256R1` elliptic curves, and they are stored in the files `hasib_identity_private.pem` and `hasib_identity_public.pem`, respectively. Here the private key is written in `PKCS#8` format and encrypted with `BestAvailableEncryption(hasib_password)`, while the public key is stored in `SubjectPublicKeyInfo` PEM format [2], [3], [12]. Finally, the message is printed to the console: "Hasib identity keys created (password: munkydgarp)", indicating that the entire process was completed successfully and the keys were stored securely.

This script is a basic but effective security verification system, which ensures security through PEM-based key management, password-protected encryption, and interactive input mechanisms. In the future, it will be securely implemented in real network environments by integrating with TLS/SSL, PKI certificate authorities, and cloud key management systems (KMS) [11], [12], [13].

Output:

```
(.venv) ahmad-badamasi@ahmad-badamasi-VMware-Virtual-Platform:~/Downloads$ python ahmad_identity_keys.py

== Hasib and Ahmad PEM file checker ==
Do not share password directly here for security.

Header of hasib_identity_private.pem:
1: -----BEGIN ENCRYPTED PRIVATE KEY-----
2: MIH0MF8GCSqGSib3DQEFDTBSMDEGCSqGSib3DQEFDDAkBBBUyLcbZ7hJD/kvq+3
Header of ahmad_identity_private.pem:
1: -----BEGIN ENCRYPTED PRIVATE KEY-----
2: MIH0MF8GCSqGSib3DQEFDTBSMDEGCSqGSib3DQEFDDAkBBBLvLSt3PPeDtiaP582

-- Hasib key load test --
Hasib private key loaded successfully (by provided password).

-- Ahmad key load test (same password tried) --
Ahmad private key is also loaded with the same password - that is, the password for both people is the same.

-- Final report --
1) Hasib known password: OK
2) Ahmad same password test: OK

Note: If unable to recover the critical key, there is no option but to re-generate the key.
```

Ahmad shared his ahmad_public.pem file with Hasib. [1], [2], [3]

Computed Shared Secret

Hasib's Shared Secret and Ephemeral Keypair generation (hasib_start_session.py)

hasib_start_session.py created and edited:

```
Hasib@Hasib MINGW64 ~/ecc_project_secure
$ touch hasib_start_session.py
(.venv)
```

```
Hasib@Hasib MINGW64 ~/ecc_project_secure
$ notepad hasib_start_session.py
(.venv)
```

Output:

```
Hasib@Hasib MINGW64 ~/ecc_project_secure
$ python hasib_start_session.py
Hasib ephemeral keypair generated & signed (salt included)
(.venv)
```

hasib_start_session.py

```
with open("hasib_ephemeral_private.pem", "wb") as f:
    f.write(ephemeral_priv.private_bytes(
        encoding=serialization.Encoding.PEM,
        format=serialization.PrivateFormat.PKCS8,
        encryption_algorithm=serialization.NoEncryption()
    ))

pkg = {
    "epub": base64.b64encode(epub_bytes).decode(),
    "signature": base64.b64encode(signature).decode(),
    "salt": base64.b64encode(salt).decode(),
    "sender": "Hasib"
}

with open("hasib_ephemeral_pub_signed.json", "w") as f:
    json.dump(pkg, f)

print("Hasib ephemeral keypair generated & signed (salt included)")
```

Ln 20, Col 1 | 1,230 characters | Plain text

```
signature = hasib_priv.sign(epub_bytes, ec.ECDSA(hashes.SHA256()))

salt = os.urandom(16)

with open("hasib_ephemeral_private.pem", "wb") as f:
    f.write(ephemeral_priv.private_bytes(
        encoding=serialization.Encoding.PEM,
        format=serialization.PrivateFormat.PKCS8,
        encryption_algorithm=serialization.NoEncryption()
    ))

pkg = {
    "epub": base64.b64encode(epub_bytes).decode(),
    "signature": base64.b64encode(signature).decode(),
    "salt": base64.b64encode(salt).decode(),
    "sender": "Hasib"
}

with open("hasib_ephemeral_pub_signed.json", "w") as f:
    json.dump(pkg, f)

print("Hasib ephemeral keypair generated & signed (salt included)")
```

Ln 29, Col 51 | 1,230 characters | Plain text

This script generates an ephemeral (temporary) elliptic curve key pair for Hasib [2], [3] and signs it with its original private key [4], [5], so that others can be sure that the ephemeral public key really came from Hasib. First, the private key is loaded from Hasib's PEM file `hasib_identity_private.pem` using `password = b"munkydgar"` [1], [6]. The ephemeral private key is then generated with `ec.generate_private_key(ec.SECP256R1())` and its public key is converted to the

SubjectPublicKeyInfo PEM format [3], [9]. This ephemeral public key is signed with Hasib's original private key using ECDSA and SHA256 [4]. A 16-byte random salt is generated for added security [3], [5].

The ephemeral private key is stored in the file `hasib_ephemeral_private.pem` in PEM format, with no encryption used [1]. The ephemeral public key, signature, and salt are then Base64 encoded and stored in a JSON package (`hasib_ephemeral_pub_signed.json`) with the "sender": "Hasib". Thus, the ephemeral key generation, signing, and packaging are completed successfully.

Output:

```
Hasib@Hasib MINGW64 ~/ecc_project_secure
$ python hasib_start_session.py
Hasib ephemeral keypair generated & signed (salt included)
(.venv)
```

(Here <base64-key-string> is the base64 version of the generated AES key, along with the Ephemeral keypair generated and signed.) [1], [2]

The `hasib_ephemeral_pub_signed.json` file was shared from Hasib to Ahmad. Ahmmad The `ahma_ephemeral_pub_signed.json` file was shared from Ahmad to Hasib.

Ahmad's Shared Secret

Ahmad's Shared Secret and Ephemeral Keypair Generation (`ahmad_start_session.py`)

`ahmad_start_session.py` was created and edited:

```
(.venv) ahmad-badamasi@ahmad-badamasi-VMware-Virtual-Platform:~/Downloads$ touch ahmad_start_session.py
```

```
(.venv) ahmad-badamasi@ahmad-badamasi-VMware-Virtual-Platform:~/Downloads$ nano ahmad_start_session.py
```

ahmad_start_session.py

```
GNU nano 7.2 ahmad_start_session.py
from cryptography.hazmat.primitives.asymmetric import ec
from cryptography.hazmat.primitives import serialization, hashes
import base64, os, json

password = b"nunkydgarp"

with open("hasib_ephemeral_pub_signed.json","r") as f:
    pkg = json.load(f)
    epub_bytes = base64.b64decode(pkg["epub"])
    signature = base64.b64decode(pkg["signature"])
    salt = base64.b64decode(pkg["salt"])

with open("hasib_identity_public.pem","rb") as f:
    hasib_identity_pub = serialization.load_pem_public_key(f.read())
    hasib_identity_pub.verify(signature, epub_bytes, ec.ECDSA(hashes.SHA256()))
    print("Hasib ephemeral signature verified")

hasib_epub = serialization.load_pem_public_key(epub_bytes)

ahmad_epriv = ec.generate_private_key(ec.SECP256R1())
ahmad_epub = ahmad_epriv.public_key()
aepub_bytes = ahmad_epub.public_bytes(
    encoding=serialization.Encoding.PEM,
    format=serialization.PublicFormat.SubjectPublicKeyInfo
)

with open("ahmad_identity_private.pem","rb") as f:
    ahmad_priv = serialization.load_pem_private_key(f.read(), password=password)
    sig_a = ahmad_priv.sign(aepub_bytes, ec.ECDSA(hashes.SHA256()))

Help Write Out Where Is Cut Execute Location M-U Undo M-A Set Mark M-] To Bracket M-Q Previous M-B Back
Exit Read File Replace Paste Justify Go To Line M-E Redo M-G Copy M-; Where Was M-W Next M-F Forward
```

```
GNU nano 7.2 ahmad_start_session.py

ahmad_epriv = ec.generate_private_key(ec.SECP256R1())
ahmad_epub = ahmad_epriv.public_key()
aepub_bytes = ahmad_epub.public_bytes(
    encoding=serialization.Encoding.PEM,
    format=serialization.PublicFormat.SubjectPublicKeyInfo
)

with open("ahmad_identity_private.pem","rb") as f:
    ahmad_priv = serialization.load_pem_private_key(f.read(), password=password)
    sig_a = ahmad_priv.sign(aepub_bytes, ec.ECDSA(hashes.SHA256()))

with open("ahmad_ephemeral_private.pem","wb") as f:
    f.write(ahmad_epriv.private_bytes(
        encoding=serialization.Encoding.PEM,
        format=serialization.PrivateFormat.PKCS8,
        encryption_algorithm=serialization.NoEncryption()
    ))

resp = {
    "epub": base64.b64encode(aepub_bytes).decode(),
    "signature": base64.b64encode(sig_a).decode(),
    "salt": base64.b64encode(salt).decode(),
    "sender": "Ahmad"
}

with open("ahmad_ephemeral_pub_signed.json","w") as f:
    json.dump(resp, f)

print("Ahmad ephemeral keypair created & signed.")
```

This script verifies Hasib's ephemeral key and generates and signs Ahmad's ephemeral key. First, Hasib's JSON file `hasib_ephemeral_pub_signed.json` is read, where the ephemeral public key (epub), signature, and salt are stored in Base64 format [1], [6]. These are decoded to obtain `epub_bytes`, signature, and salt, respectively. Then Hasib's original public key is loaded from

hasib_identity_public.pem [1], [9] and the ephemeral signature is verified using the verify function [2], [3], [4]. If successful, the console prints ephemeral signature verified.

Ahmad's ephemeral key is then generated using `ec.generate_private_key(ec.SECP256R1())` and his public key is converted to PEM format [3], [9]. Ahmad's private key is loaded using the password from `ahmad_identity_private.pem` [1], and the ephemeral public key is signed with ECDSA and SHA256 [4], [5]. Ahmad's ephemeral private key is stored in PEM format (without any encryption) in `ahmad_ephemeral_private.pem` [1]. The ephemeral public key, signature, and Hasib's salt are then Base64 encoded into a JSON package (`ahmad_ephemeral_pub_signed.json`) with "sender": "Ahmad" [1], [6].

Output:

```
(.venv) ahmad-badamasi@ahmad-badamasi-VMware-Virtual-Platform:~/Downloads$ python ahmad_start_session.py
Hasib ephemeral signature verified
Ahmad ephemeral keypair created & signed.
```

Both get the same AES key (base64 output will match) - as a result of ECDH and HKDF processes. Also, Ephemeral Keypair was created with signed and Signature was verified. [4], [5], [9]

The `ahmad_ephemeral_pub_signed.json` file was shared from Ahmad to Hasib successfully.

Encrypted & Decrypted Messages with AES

Hasib encrypts messages (`hasib_encrypt.py`)

`hasib_encrypt.py` was created:

```
Hasib@Hasib MINGW64 ~/ecc_project_secure
$ touch hasib_encrypt.py
(.venv)
```

```
Hasib@Hasib MINGW64 ~/ecc_project_secure
$ notepad hasib_encrypt.py
(.venv)
```


hasib_encrypt.py

```
File Edit View

import json, base64, os, time
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
from cryptography.hazmat.primitives import serialization, hashes
from cryptography.hazmat.primitives.kdf.hkdf import HKDF
from cryptography.hazmat.primitives.asymmetric import ec

with open("hasib_ephemeral_private.pem", "rb") as f:
    hasib_epriv = serialization.load_pem_private_key(f.read(), password=None)

with open("ahmad_ephemeral_pub_signed.json", "r") as f:
    resp = json.load(f)
aepub_bytes = base64.b64decode(resp["epub"])
salt = base64.b64decode(resp["salt"])
ahmad_epub = serialization.load_pem_public_key(aepub_bytes)

shared = hasib_epriv.exchange(ec.ECDH(), ahmad_epub)
derived_key = HKDF(
    algorithm=hashes.SHA256(),
    length=32,
    salt=salt,
    info=b'handshake data'
).derive(shared)

plaintext = b"Hi brother, this is the secret message from Hasib. The world id now running on computing power and the power of the cloud. We'll change the world and enhance the highest power of computing. Our PRAM and PCM devices will be faster in the picosecond range and will be more secure. I am sending you encrypted message using AES-256 GCM!"
aad_data = {
    "msg_id": "msg2025",
    "sender": "Hasib",
    "timestamp": str(int(time.time()))
}
```

```
nonce = os.urandom(12)
aesgcm = AESGCM(derived_key)
ciphertext = aesgcm.encrypt(nonce, plaintext, aad)

pkg = {
    "salt": base64.b64encode(salt).decode(),
    "nonce": base64.b64encode(nonce).decode(),
    "aad": base64.b64encode(aad).decode(),
    "ciphertext": base64.b64encode(ciphertext).decode()
}

with open("message_secure.json", "w") as f:
    json.dump(pkg, f)

print("Message encrypted successfully! (file: message_secure.json)")
```

Ln 31, Col 1

1,735 characters

Plain text

This script creates a secure encrypted message from Hasib to Ahmad. It uses elliptic-curve Diffie-Hellman (ECDH) based session key generation and AES 256 GCM encryption to keep the message private, authenticated, and unalterable. First, Hasib loads his own ephemeral private key from `hasib_ephemeral_private.pem`, and reads the `ahmad_ephemeral_pub_signed.json` file received from Ahmad and Base64 decodes Ahmad's ephemeral public key (`epub`) and salt (for PEM/serialization and client-library purposes). [1]

The two ephemeral keys are then used to create a shared secret using `hasib_epriv.exchange(ec.ECDH(), ahmad_epub)` - an ECDH process that only both parties can read; The specific protocol/security rules for this step are consistent with the principles described in SP 800 56A. [5]

A `derived_key` (AES 256 key) of 32 bytes is generated by applying HKDF (HMAC based Key Derivation Function) to this shared secret using SHA 256 and the given salt - the design and security concept of HKDF is described in the HKDF article. [4]

The plaintext then contains Hasib's original secret message, which is encrypted with AES GCM. An AAD (Additional Authenticated Data) is generated with the message - it contains the message ID, sender name and timestamp and is encoded as JSON bytes. A 12-byte random nonce is generated for security, and the ciphertext is obtained by calling `AESGCM(derived_key).encrypt(nonce, plaintext, aad)` The AES GCM operation and its authentication/integrity model are compliant with NIST guidelines. [6], [7]

Finally, the salt, nonce, aad, and ciphertext are Base64 encoded and packaged into a JSON object (`message_secure.json`) - thus the message is securely encrypted and stored using Hasib and Ahmad's ephemeral key-based session key. [5], [6]

Output:

```
Hasib@Hasib MINGW64 ~/ecc_project_secure
$ python hasib_encrypt.py
Message encrypted successfully! (file: message_secure.json)
Hasib@Hasib MINGW64 ~/ecc_project_secure
$ |
```

The content of the `message_secure.json` file will contain the nonce, aad, ciphertext, salt in base64 format. [4], [7]

`Message_secure.json` file was shared to Ahmad for decryption process.

Ahmad decrypts the message (ahmad_decrypt.py)

ahmad_decrypt.py is created:

```
(.venv) ahmad-badamsi@ahmad-badamsi-VMware-Virtual-Platform:~/Downloads$ touch ahmad_decrypt.py
```

```
(.venv) ahmad-badamsi@ahmad-badamsi-VMware-Virtual-Platform:~/Downloads$ nano ahmad_decrypt.py
```

ahmad_decrypt.py

```
GNU nano 7.2 ahmad_decrypt.py
import json, base64
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
from cryptography.hazmat.primitives import serialization, hashes
from cryptography.hazmat.primitives.kdf.hkdf import HKDF
from cryptography.hazmat.primitives.asymmetric import ec

with open("ahmad_ephemeral_private.pem", "rb") as f:
    ahmad_epriv = serialization.load_pem_private_key(f.read(), password=None)

with open("hasib_ephemeral_pub_signed.json", "r") as f:
    hpkg = json.load(f)
    hepub_bytes = base64.b64decode(hpkg["epub"])
    salt = base64.b64decode(hpkg["salt"])
    hasib_epub = serialization.load_pem_public_key(hpub_bytes)

shared = ahmad_epriv.exchange(ec.ECDH(), hasib_epub)
derived_key = HKDF(
    algorithm=hashes.SHA256(),
    length=32,
    salt=salt,
    info=b'handshake data'
).derive(shared)

with open("message_secure.json", "r") as f:
    msgpkg = json.load(f)
    nonce = base64.b64decode(msgpkg["nonce"])
    aad = base64.b64decode(msgpkg["aad"])
    ciphertext = base64.b64decode(msgpkg["ciphertext"])

aesgcm = AESGCM(derived_key)
```

```
GNU nano 7.2 ahmad_decrypt.py
with open("ahmad_ephemeral_private.pem", "rb") as f:
    ahmad_epriv = serialization.load_pem_private_key(f.read(), password=None)

with open("hasib_ephemeral_pub_signed.json", "r") as f:
    hpkg = json.load(f)
    hepub_bytes = base64.b64decode(hpkg["epub"])
    salt = base64.b64decode(hpkg["salt"])
    hasib_epub = serialization.load_pem_public_key(hpub_bytes)

shared = ahmad_epriv.exchange(ec.ECDH(), hasib_epub)
derived_key = HKDF(
    algorithm=hashes.SHA256(),
    length=32,
    salt=salt,
    info=b'handshake data'
).derive(shared)

with open("message_secure.json", "r") as f:
    msgpkg = json.load(f)
    nonce = base64.b64decode(msgpkg["nonce"])
    aad = base64.b64decode(msgpkg["aad"])
    ciphertext = base64.b64decode(msgpkg["ciphertext"])

aesgcm = AESGCM(derived_key)
plaintext = aesgcm.decrypt(nonce, ciphertext, aad)

print("\n Decryption successful!")
print("AAD:", aad.decode())
print("Plaintext message:\n", plaintext.decode())
```

This script is used to successfully decrypt the encrypted message sent by Ahmad to Hasib. Here, the Elliptic Curve Diffie-Hellman (ECDH) and AES-256-GCM decryption process is followed.

First, Ahmad loads his ephemeral private key from `ahmad_ephemeral_private.pem` and reads the JSON file created by Hasib, `hasib_ephemeral_pub_signed.json`, and decodes the ephemeral public key (epub) and salt (salt) from Base64 [1], [6]. Then, Hasib's ephemeral public key is loaded with `serialization.load_pem_public_key()` [1], [9].

Then, using Ahmad's ephemeral private key and Hasib's ephemeral public key, a shared secret is created using `exchange(ec.ECDH(), hasib_epub)`, which is the same for both parties [5]. Then, HKDF (HMAC-based Key Derivation Function) is used to generate a 32-byte `derived_key` (AES-256 key) using SHA-256 algorithm, salt, and `info=b'handshake data'` [4], [5]. Then, Hasib's encrypted message package is read from the `message_secure.json` file, which contains the nonce (12-byte random number), aad (additional authentication data), and ciphertext (the original encrypted message). These are decoded from Base64 and placed in the appropriate variables [6].

Finally, an AES-GCM object is created using `AESGCM(derived_key)` and the original message is decrypted using the `decrypt(nonce, ciphertext, aad)` call. As a result, Ahmad Hasib's message is successfully decrypted and the integrity and authenticity of the message are also verified [6], [7].

Output:

```
(.venv) ahmad-badanasi@ahmad-badanasi-VMware-Virtual-Platform:~/Downloads$ python ahmad_decrypt.py

Decryption successful!
AAD: {"msg_id": "msg2025", "sender": "Hasib", "timestamp": "1760867734"}
Plaintext message:
Hi brother, this is the secret message from Hasib. The world id now running on computing power and the power of the cloud. We'll change the world and enhance the highest power of computing. Our PRAM and PCM devices will be faster in the picosecond range and will be more secure. I am sending you encrypted message using AES-256 GCM!
```

Decryption successful!

AAD: {"msg_id": "msg2025", "sender": "Hasib", "timestamp": "1760824099"}

Plaintext message:

Hi brother, this is the secret message from Hasib. The world id now running on computing power and the power of the cloud. We'll change the world and enhance the highest power of computing. Our PRAM and PCM devices will be faster in the picosecond range and will be more secure. I am sending you encrypted message using AES-256 GCM!

Here timestamp is shown as an example from the reference document - in reality it would be a session-dependent Unix timestamp. [1], [6], [11]

Security Development and Implementation

This project uses modern cryptographic techniques for security development and implementation. Operating system-level permissions (chmod 600) are used to protect the private key, ensuring file-level security [1]. AES encryption and the BestAvailableEncryption(password) method are used for password-protected PEM files, where the password is munkydgarp; this ensures that the identity key is securely stored and prevents unauthorized access [6]. The HKDF (HMAC-based Key Derivation Function) is used to increase the strength of the key derivation, where the random salt is generated by os.urandom(16) [4]. This increases security by reducing the risk of key duplication. The ephemeral key mechanism generates a new key pair for each session, ensuring “forward secrecy” - that is, if a one-time key is leaked, previous messages remain secure [2], [5]. To verify the authenticity of the message, Hashib signs and Ahmad verifies it, which helps prevent man-in-the-middle (MITM) attacks. In addition, Associated Data (AAD) is added to each message - such as the message ID, sender name, and timestamp - to maintain message integrity [7], [8]. To prevent nonce reuse, a new nonce is generated each time using os.urandom(12), which ensures cryptographic security in AES-GCM mode [7].

System Evaluation and Future Development

System Strengths:

The project implements full asymmetric key-based authentication, where authenticated encryption is used in AES-GCM mode. Forward secrecy is achieved through the use of salts and ephemeral keys, and the simple message exchange system in JSON format is lightweight and efficient [1], [6], [7].

Limitations and room for improvement:

Currently, the password is hardcoded in the script; in the future, getpass() should be used for increased security [10]. Additionally, adding a PKI (Public Key Infrastructure) or Certificate Authority will increase the reliability and authentication strength of the system [11]. Integration with TLS/SSL (Transport Layer Security) protocols will make it suitable for use in real network environments [12].

Future plans:

Future plans include dynamic password input, certificate-based verification, and cloud-secure key exchange services (such as AWS KMS) [9], [13]. These improvements will make the system more scalable, reliable, and compatible with industry-standard security frameworks.

Conclusion

The entire framework of this project is based on modern cryptographic technology, where a balance between security, reliability, and practicality is maintained. The system is primarily designed for secure message exchange between two users: Hasib and Ahmad - where strong encryption, key management, and authentication are implemented at each level.

The key generation and signing process is kept very lightweight and efficient by using Elliptic Curve Cryptography (ECC). The SECP256R1 curve provides a high level of security, while also being relatively low in computational cost [2]. The use of ephemeral keypairs results in new keys being generated for each session, which ensures “Forward Secrecy” - that is, even if the key of a session is leaked, the confidentiality of previous or future messages remains intact [5]. The AES-GCM (Advanced Encryption Standard - Galois/Counter Mode) is used for message encryption and authentication, which provides both confidentiality and integrity at the same time. In AES-GCM mode, a new 12-byte nonce is used each time by `os.urandom(12)`, which completely eliminates the possibility of threats arising from nonce repetition [7], [8]. A strong 32-byte session key with a random salt is generated by applying HKDF (HMAC-based Key Derivation Function), which creates a secure channel derived from the shared ephemeral key of both parties [4].

Additionally, message metadata such as message ID, sender name, and timestamp are added as Associated Data (AAD), which helps to verify the authenticity of the message through AES-GCM authentication and makes it possible to detect any modification or forgery [6]. In the signature verification system, Hasib signed the message with his private key and Ahmad verified it with his public key, which prevents any man-in-the-middle (MITM) attacker from modifying the message.

However, there are some limitations to the project. Since the password is hardcoded, this may pose a security risk when implemented in a real system; In the future, the use of `getpass()` or a secure password input mechanism will reduce this risk [10]. Moreover, the credibility and institutional

acceptance of the system will be greatly increased by connecting a Certificate Authority (CA) or PKI (Public Key Infrastructure) [11]. If support for the TLS/SSL protocol is added in a more advanced version, it will also be usable for Internet-scale communication [12].

Future developments are planned - implementing cloud-based key management (such as AWS KMS) [13], certificate-based authentication, and dynamic password input methods. These steps will move the project from being just an academic exercise, to a real industry-standard secure communication framework.

References

- [1] “cryptography - Python Package Index (PyPI),” *PyPI.org*. Available: <https://pypi.org/project/cryptography/>
- [2] D. Bernstein, T. Lange, and P. Schwabe, “The security impact of elliptic curve choice,” *IACR Cryptology ePrint Archive*, vol. 2016, pp. 1–21, 2016. Available: <https://eprint.iacr.org/2016/001.pdf>
- [3] A. Menezes, P. van Oorschot, and S. Vanstone, *Handbook of Applied Cryptography*. CRC Press, 1996. Available: <https://cacr.uwaterloo.ca/hac/>
- [4] H. Krawczyk, “Cryptographic extraction and key derivation: The HKDF scheme,” *IACR ePrint Archive*, 2010/264, 2010. Available: <https://eprint.iacr.org/2010/264.pdf>
- [5] NIST, “Recommendation for Pair-Wise Key Establishment Using Discrete Logarithm Cryptography (SP 800-56A Rev. 3),” *National Institute of Standards and Technology*, 2018. DOI: 10.6028/NIST.SP.800-56Ar3
- [6] “Cryptography.io: AESGCM,” *Cryptography Documentation*. Available: <https://cryptography.io/en/latest/hazmat/primitives/symmetric-encryption/#aesgcm>
- [7] NIST, “Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC,” *NIST Special Publication 800-38D*, 2007. DOI: 10.6028/NIST.SP.800-38D
- [8] D. J. Bernstein, “The Security of Galois/Counter Mode (GCM),” *IACR ePrint Archive*, 2005. Available: <https://eprint.iacr.org/2005/141.pdf>
- [9] “Installation - Cryptography,” *cryptography.io Documentation*. Available: <https://cryptography.io/en/latest/installation/>
- [10] Python Software Foundation, *getpass - Portable Password Input*, Python 3 Documentation. Available: <https://docs.python.org/3/library/getpass.html>
- [11] R. Kuhn, “Introduction to Public Key Technology and the Federal PKI Infrastructure,” *NIST*

Special Publication 800-32, 2001. Available:

<https://nvlpubs.nist.gov/nistpubs/legacy/sp/nistspecialpublication800-32.pdf>

[12] E. Rescorla, “The Transport Layer Security (TLS) Protocol Version 1.2,” *RFC 5246*, 2008.

Available: <https://datatracker.ietf.org/doc/html/rfc5246>

[13] Amazon Web Services, Inc., *AWS Key Management Service Cryptographic Details*, 2020.

Available: <https://docs.aws.amazon.com/pdfs/kms/latest/cryptographic-details/kms-cryptographic-details.pdf>